Large Language Models – IASD App

Cours 1

March 9, 2026

Shuyu Dong

Dauphine | PSL UNIVERSITÉ PARIS LAMSADE UMR 7243

Course Plan

- Machine learning for natural language processing (NLP)
- Attention and transformers
- Transformers and pre-training
- TP: GPT
- TP: GPT (Part 2), BERT
- Evaluation of LLMs

Evaluation:

- 2 homeworks
- 1 graded TP
- 1 project

References and sources:

- Course slides on LLMs (IASD) by Alexandre Allauzen and Florian Le Bronnec.
- Other references to be detailed.

Plan du cours

- 1.1 – LLMs: applications, number of parameters
- 1.2 – Machine learning for natural language processing (NLP)
- 1.3 – TP: IMDB dataset, word embeddings

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

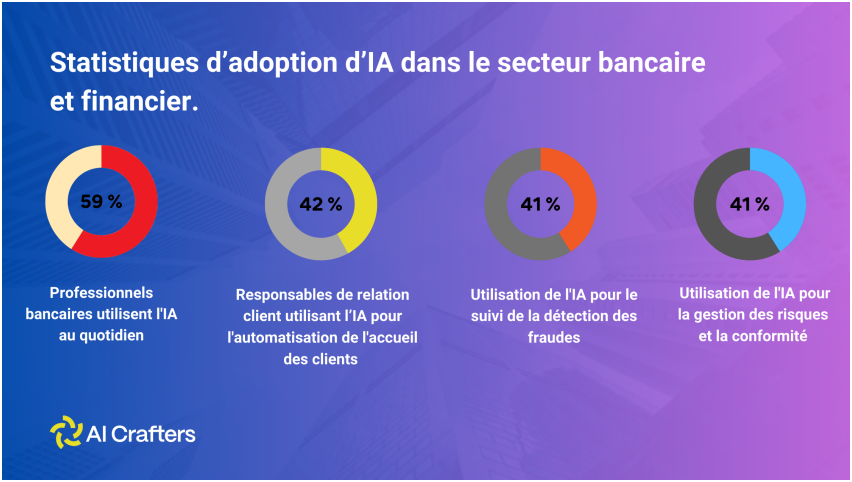
2.1 – Word embeddings for text classification

2.2 – Word2vec

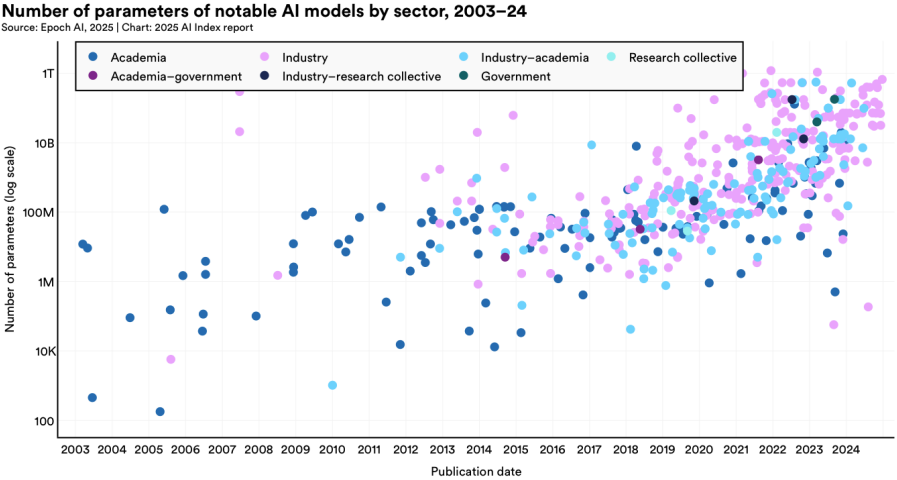
2.3 – TP: Word embeddings for text classification (IMDB dataset)

Large language models: applications

Large language models: applications

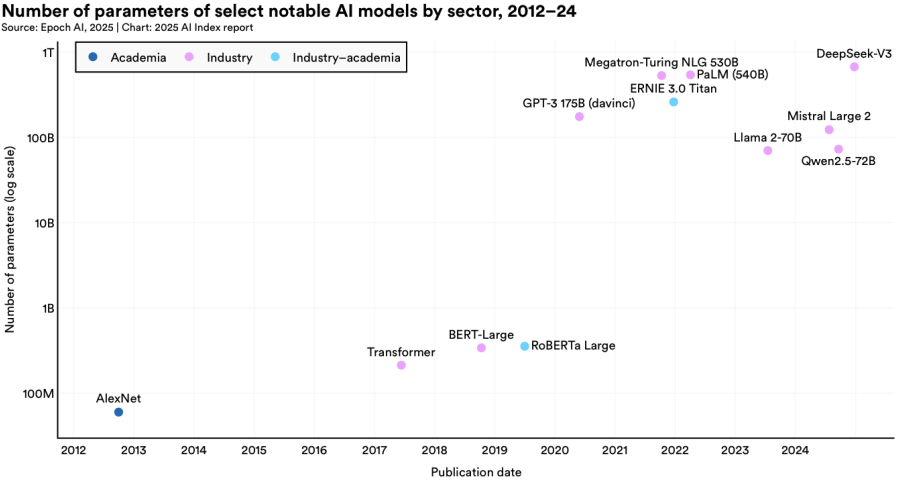


Large language models: number of parameters



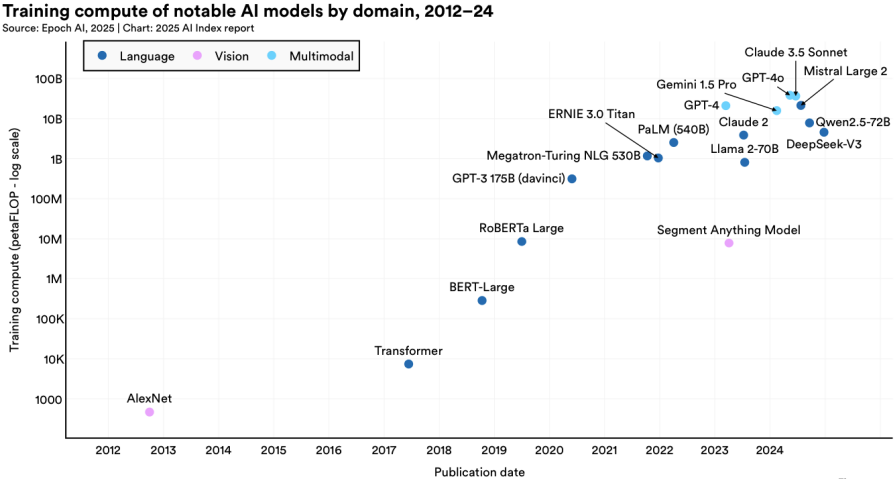
Source: https://hai.stanford.edu/ai-index/2025-ai-index-report

Large language models: number of parameters



Source: https://hai.stanford.edu/ai-index/2025-ai-index-report

Large language models: training compute



Source: https://hai.stanford.edu/ai-index/2025-ai-index-report

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

2.1 – Word embeddings for text classification

2.2 – Word2vec

2.3 – TP: Word embeddings for text classification (IMDB dataset)

Natural language

The goal of natural language (human language) is communication.

- For the speaker:
  - Intention: what is the message?
  - Generation: transform the message into a sequence.
  - Synthesis: output the sequence (speech, written text, signs, . . .)
- For the hearer:
  - Perception
  - Analysis
  - Incorporation

Natural language processing: processing of natural language by a computer.

Natural language processing (NLP): challenges

Implicity: natural language is implicit, which enables conciseness (ellipsis, anaphora, . . .), but implicity entails potential **ambiguities**.

Why does this happen for natural language?

- Shared world knowledge
- Cognitive efficiency
- Social pragmatics

Having a unique linguistic expression for every possible conceptualization would make language overly complex and linguistic expressions unnecessarily long.
(R. Mooney)

Natural language processing (NLP): challenges

Why is natural language ambiguous?

- Lexical / word-level reasons: synonymes (e.g., bank, plant, . . .)
- Structural ambiguity (syntax):
  e.g., *"I saw the man on the hill with a telescope."*
  (From the course of R. Mooney)
- Referential ambiguity:
  e.g., *"Amy told Anna that she was late."*
- Context dependence:
  e.g., *"It's over there."*

This is hard for machines and for AI:

- No shared knowledge/experience (or lack of shared knowledge for AIs)
- No physical grounding
- No social intuition

NLP is a starting point for machines to understand natural language.

Machine learning for NLP

Model NLP tasks as machine learning problems:

- Text classification (our focus today)

$$X = (w_1, w_2, \ldots, w_L) \rightarrow y(\text{class label})$$

- Word tagging

$$X = (w_1, w_2, \ldots, w_L) \rightarrow (y_1, y_2, \ldots, y_L)$$

Text classification

A supervised learning problem:

Training data:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

Test data:

"My wonderful friend took me to see this movie for our anniversary, it was terrible."

→ negative or positive?

Questions:

- How to represent the input text?
  Unlike images, sounds or other numerical signals, natural language **does not** have straightforward representations in a vector space.
- How to perform classification?

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

2.1 – Word embeddings for text classification

2.2 – Word2vec

2.3 – TP: Word embeddings for text classification (IMDB dataset)

Large Language Models – IASD App

Cours 2

March 11, 2026

Shuyu Dong

Dauphine | PSL UNIVERSITÉ PARIS LAMSADE UMR 7243

Plan du cours

- 2.1 – Word embeddings for text classification
- 2.2 – Word2vec
- 2.3 – TP: Word embeddings for text classification (IMDB dataset)

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

2.1 – Word embeddings for text classification

2.2 – Word2vec

2.3 – TP: Word embeddings for text classification (IMDB dataset)

Text classification using word counts

Train:

- Build document-term matrix $C \in \mathbb{R}^{n \times V}$ from training texts:

$$\begin{array}{c|cccc} & w_1 & w_2 & \dots & w_V \\ \hline t_1 & C_{11} & C_{12} & & C_{1V} \\ t_2 & C_{21} & C_{22} & & C_{2V} \\ \vdots & & & & \\ t_n & C_{n1} & C_{n2} & & C_{nV} \end{array}$$

V: vocabulary size
n: number of documents (e.g., reviews)

- Consider word count by class $S = W_y{}^{\mathrm{T}} C \in \mathbb{R}^{2 \times V}$, where W_y is the class indicator matrix:

$$W_y{}^{\mathrm{T}} = \left(\begin{array}{c|cccc} & t_1 & t_2 & \dots & t_n \\ \hline \text{Class “1”} & 0 & 1 & \dots & 1 \\ \text{Class “2”} & 1 & 0 & \dots & 0 \end{array} \right) \Rightarrow \quad S = \left(\begin{array}{c|ccc} & \dots & w_j & \dots \\ \hline \text{Class “1”} & \dots & \sum_{k \text{ s.t. } t_k \in \text{Class “1”}} C_{kj} & \dots \\ \text{Class “2”} & \dots & \sum_{k \text{ s.t. } t_k \in \text{Class “2”}} C_{kj} & \dots \end{array} \right)$$

Test:

- For a test document t: compute the score of “t in class 1 (resp. 2)”

$$s(1, t) = \sum_{w \in t} S(1, w), \quad s(2, t) = \sum_{w \in t} S(2, w).$$

Finally, get prediction value $f(t) = \arg\max_i (s(i, t))$.

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.

    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.

    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.

    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

- Determine the prediction function $f_\theta : \mathcal{T} \to \mathcal{Y}$: $\hat{y} = f_\theta(T)$.
  - Example: $f_\theta(T) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_\theta(k, T).$

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.

    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

- Determine the prediction function $f_\theta : \mathcal{T} \to \mathcal{Y}$: $\hat{y} = f_\theta(T)$.
  - Example: $$f_\theta(T) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\ p_\theta(k, T).$$

Questions:

- How to define a parametric function $p_\theta(y, T)$ on $\mathcal{Y} \times \mathcal{T}$ to model $P(y|T)$?

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.

    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

- Determine the prediction function $f_\theta : \mathcal{T} \to \mathcal{Y}$: $\hat{y} = f_\theta(T)$.
  - Example: $f_\theta(T) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_\theta(k, T)$.

Questions:

- How to define a parametric function $p_\theta(y, T)$ on $\mathcal{Y} \times \mathcal{T}$ to model $P(y|T)$?
  - Remark: a vector representation of T is needed to interact with parameters θ.

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.
    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

- Determine the prediction function $f_\theta : \mathcal{T} \to \mathcal{Y}$: $\hat{y} = f_\theta(T)$.
  - Example: $f_\theta(T) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_\theta(k, T)$.

Questions:

- How to define a parametric function $p_\theta(y, T)$ on $\mathcal{Y} \times \mathcal{T}$ to model $P(y|T)$?
  - Remark: a vector representation of T is needed to interact with parameters θ.
  - Preliminary question: *How to map T into a vector space?* (word embedding)

Text classification using word embeddings

Supervised learning framework:

- Random variables: $(T, y) \in \mathcal{T} \times \mathcal{Y}$
  - $\mathcal{T}$: space of texts, each text is a sequence of words.
  - Note: the length (number of words) of any text in $\mathcal{T}$ is bounded by a constant L.
    (Related questions: padding, TP1)
  - $\mathcal{Y} \simeq \{1, \ldots, K\}$: set of labels
- Training data: $\mathcal{D} = \{T_i, y_i\}_{i=1,\ldots,n}$ with $T_i \in \mathcal{T}$ and $y_i \in \mathcal{Y}$.
- Model $P(y|T)$ with a set of parameters θ:

$$P(y|T) = p_\theta(y, T).$$

- Determine the prediction function $f_\theta : \mathcal{T} \to \mathcal{Y}$: $\hat{y} = f_\theta(T)$.
  - Example: $f_\theta(T) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_\theta(k, T)$.

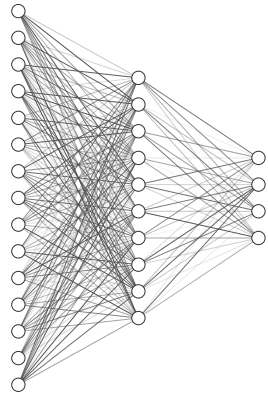
Questions:

- How to define a parametric function $p_\theta(y, T)$ on $\mathcal{Y} \times \mathcal{T}$ to model $P(y|T)$?
  - Remark: a vector representation of T is needed to interact with parameters θ.
  - Preliminary question: *How to map T into a vector space?* (word embedding)
- How to define the training objective?
  - Loss function: $L(\hat{y}, y)$
  - Empirical risk: $R(\theta; \mathcal{D}) = \sum_{i=1}^{n} L(\hat{y}_i, y_i)$.

Text classification: word embeddings

- **Word embedding matrix** R. For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$:
  $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Text classification: word embeddings

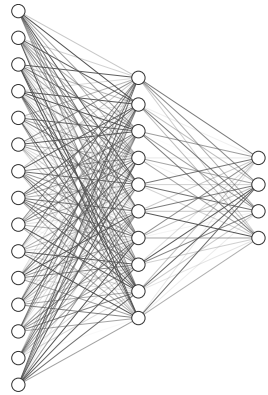
- **Word embedding matrix** R. For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$: $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$

- **Prediction function:** $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ where $\sigma(\cdot)$ is the sigmoid or the softmax function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma((z_1, \ldots, z_K)) = \left(\frac{e^{z_1}}{\sum_i e^{z_i}}, \ldots, \frac{e^{z_K}}{\sum_i e^{z_i}}\right).$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Text classification: word embeddings

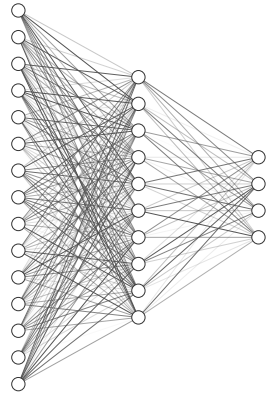
- **Word embedding matrix** R. For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$:
  $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$

- **Prediction function:** $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ where $\sigma(\cdot)$ is the sigmoid or the softmax function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma((z_1, \ldots, z_K)) = \left(\frac{e^{z_1}}{\sum_i e^{z_i}}, \ldots, \frac{e^{z_K}}{\sum_i e^{z_i}}\right).$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Training objective:

- Model parameters: $\theta = (R, W)$.
- Empirical risk (multiclass case $K \geq 3$):
  $L(\theta) = -\sum_{i=1}^{N_{\text{train}}} \sum_{k=1}^{K} y_k^{(i)} \log(\hat{y}_k^{(i)})$, for $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ with softmax.

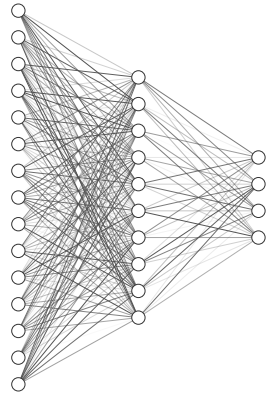
Text classification: word embeddings

- **Word embedding matrix** R**.** For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$:
  $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$

- **Prediction function:** $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ where $\sigma(\cdot)$ is the sigmoid or the softmax function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma((z_1, \ldots, z_K)) = \left(\frac{e^{z_1}}{\sum_i e^{z_i}}, \ldots, \frac{e^{z_K}}{\sum_i e^{z_i}}\right).$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Text lengths: handle different text lengths without truncation

- Build $X \in \mathbb{R}^{b \times L \times d}$ where L is the maximal length of the texts in the batch (of b samples).

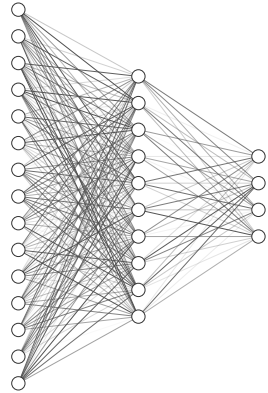
Text classification: word embeddings

- **Word embedding matrix** R. For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$:
  $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$

- **Prediction function:** $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ where $\sigma(\cdot)$ is the sigmoid or the softmax function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma((z_1, \ldots, z_K)) = \left(\frac{e^{z_1}}{\sum_i e^{z_i}}, \ldots, \frac{e^{z_K}}{\sum_i e^{z_i}}\right).$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Text lengths: handle different text lengths without truncation

- Build $X \in \mathbb{R}^{b \times L \times d}$ where L is the maximal length of the texts in the batch (of b samples).
- For texts shorter than the longest text: append padding values to fill up to length L

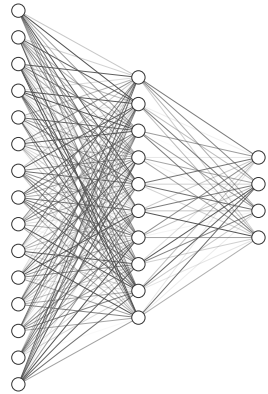
Text classification: word embeddings

- **Word embedding matrix** R. For each text $T \in \mathcal{T}$, build $X \in \mathbb{R}^{L \times d}$:
  $X_{i,:} \leftarrow$ look up $w_i \in \mathcal{T}$ in the embedding matrix $R \in \mathbb{R}^{V \times d}$.
- **Text embedding:** aggregation by mean pooling over the text's words

$$\tilde{x}^{\mathrm{T}} \leftarrow \frac{1}{L} \sum_{i=1}^{L} X_{i,:} = \frac{1}{L} \mathbf{1}^{\mathrm{T}} X$$

- **Prediction function:** $\hat{y} = \sigma(W^{\mathrm{T}} \tilde{x})$ where $\sigma(\cdot)$ is the sigmoid or the softmax function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma((z_1, \ldots, z_K)) = \left(\frac{e^{z_1}}{\sum_i e^{z_i}}, \ldots, \frac{e^{z_K}}{\sum_i e^{z_i}}\right).$$



$$\mathcal{T} \xrightarrow{R,\text{pooling}} \tilde{x} \xrightarrow{W,\sigma} \sigma(W^{\mathrm{T}} \tilde{x})$$

Text lengths: handle different text lengths without truncation

- Build $X \in \mathbb{R}^{b \times L \times d}$ where L is the maximal length of the texts in the batch (of b samples).
- For texts shorter than the longest text: append padding values to fill up to length L
- Update text embedding rule: for each sample $X \in \mathbb{R}^{L \times d}$, $\tilde{x} = \frac{1}{L_i} \sum_{i=1}^{L} X_{i,:}$ where L_i is the real length of text T_i.

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

2.1 – Word embeddings for text classification

2.2 – Word2vec

2.3 – TP: Word embeddings for text classification (IMDB dataset)

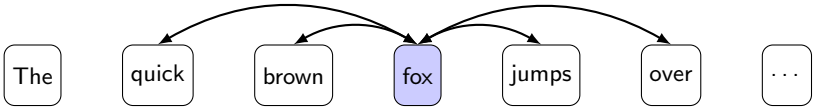
Word2vec: word embedding with contextualization

Word2vec (Mikolov et al., 2013a,b):

- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013a). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems, 26.

Word2vec: word embedding with contextualization

Context window:

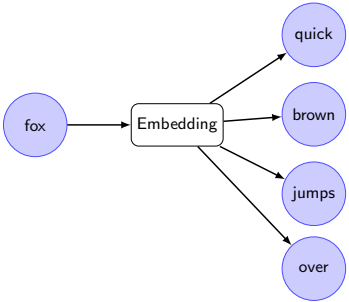


Skip-gram model:

- A collection $\mathcal{T}$ of texts, each text is a sequence of words.
- Vocabulary: $\mathcal{V}$ of V words.
- Context window size: c

For each word w_j that is considered as a center word, we want to maximize the likelihood of the context words of w_j given w_j:

$$P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j).$$



Word2vec: contextualization by the skip-gram model

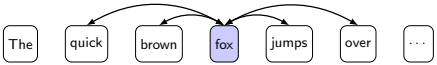
Goal: maximize $P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j)$.

- Embedding matrix $W \in \mathbb{R}^{V \times d}$ (to train)
- For each word w: encode w with an one-hot vector

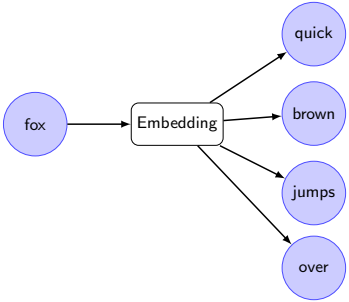
$$x_w \in \mathbb{R}^V, (x_w)_i = \begin{cases} 1 & \text{if token id of w is } i \\ 0 & \text{otherwise} \end{cases} .$$

- Word embedding: $v_w = W^{\mathrm{T}} x_w$.

Context window:



Skip-gram model:



Word2vec: contextualization by the skip-gram model

Goal: maximize $P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j)$.

- Embedding matrix $W \in \mathbb{R}^{V \times d}$ (to train)
- For each word w: encode w with an one-hot vector

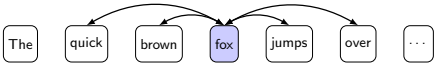
$$x_w \in \mathbb{R}^V, (x_w)_i = \begin{cases} 1 & \text{if token id of w is } i \\ 0 & \text{otherwise} \end{cases}.$$

- Word embedding: $v_w = W^{\mathrm{T}} x_w$.
- For context words $w_{j+\ell}$ of word w_j, define $P(w_{j+\ell} | w_j)$ with softmax:

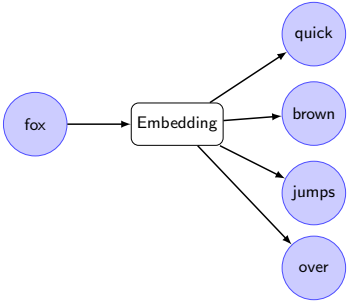
$$P(w_{j+\ell} | w_j) = \frac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{V}} \exp({v_w}^{\mathrm{T}} v_{w_j})}.$$

(based on the whole vocabulary)

Context window:



Skip-gram model:



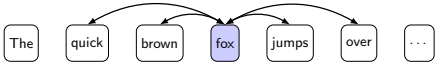
Word2vec: contextualization by the skip-gram model

Goal: maximize $P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j)$.

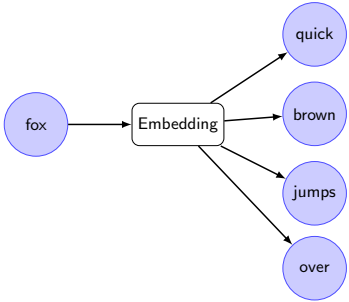
- Word embedding: $v_w = W^{\mathrm{T}} x_w$.
- For context words $w_{j+\ell}$ of word w_j, define $P(w_{j+\ell} | w_j)$ with softmax:

$$P(w_{j+\ell} | w_j) = \frac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{V}} \exp({v_w}^{\mathrm{T}} v_{w_j})} \quad \text{(whole vocab.)}$$

Context window



Skip-gram model:



Word2vec: contextualization by the skip-gram model

Goal: maximize $P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j)$.

- Word embedding: $v_w = W^{\mathrm{T}} x_w$.
- For context words $w_{j+\ell}$ of word w_j, define $P(w_{j+\ell}|w_j)$ with softmax:

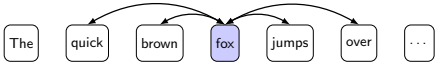
$$P(w_{j+\ell}|w_j) = \frac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{V}} \exp({v_w}^{\mathrm{T}} v_{w_j})} \quad \text{(whole vocab.)}$$

- $P(w_{j+\ell}|w_j) = \dfrac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{N} \cup \mathcal{P}} \exp({v_w}^{\mathrm{T}} v_{w_j})}$ (negative sampling)

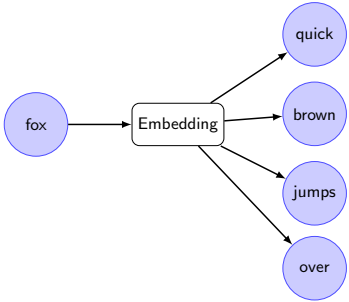
where

- $\mathcal{N}$ contains a number of words sampled randomly and independently of w_j.
- $\mathcal{P}$ contains the context words of w_j.

Context window



Skip-gram model:



Word2vec: contextualization by the skip-gram model

Goal: maximize $P(w_{j-c}, \ldots, w_{j-1}, w_{j+1}, \ldots, w_{j+c} | w_j)$.

- Word embedding: $v_w = W^{\mathrm{T}} x_w$.
- For context words $w_{j+\ell}$ of word w_j, define $P(w_{j+\ell}|w_j)$ with softmax:

$$P(w_{j+\ell}|w_j) = \frac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{V}} \exp({v_w}^{\mathrm{T}} v_{w_j})} \quad \text{(whole vocab.)}$$

- $P(w_{j+\ell}|w_j) = \dfrac{\exp({v_{w_{j+\ell}}}^{\mathrm{T}} v_{w_j})}{\sum_{w \in \mathcal{N} \cup \mathcal{P}} \exp({v_w}^{\mathrm{T}} v_{w_j})}$ (negative sampling)

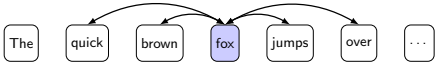
where

- $\mathcal{N}$ contains a number of words sampled randomly and independently of w_j.
- $\mathcal{P}$ contains the context words of w_j.

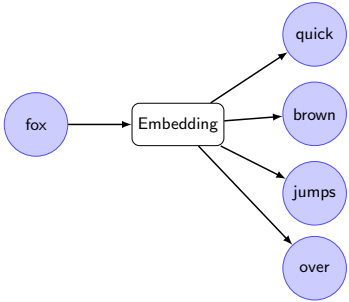
- Maximize the log-likelihood over training data $\mathcal{T}$:

$$\mathcal{L}(W) = \sum_{T \in \mathcal{T}, w_j \in T} \sum_{-c \le \ell \le c, \ell \ne 0} \log P(w_{j+\ell}|w_j).$$

Context window



Skip-gram model:



Word2vec: contextualization by the skip-gram model

(Mikolov et al., 2013b):

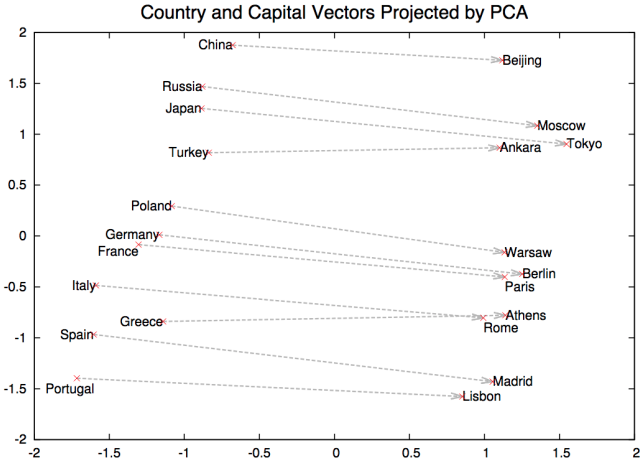


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

Plan du cours

1.1 – LLMs: applications, number of parameters

1.2 – Machine learning for natural language processing (NLP)

1.3 – TP: IMDB dataset, word embeddings

2.1 – Word embeddings for text classification

2.2 – Word2vec

2.3 – TP: Word embeddings for text classification (IMDB dataset)

Thank You !

References

Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013a). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26.